DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The CLI To-Do List Manager is a Python-based command-line application designed for efficient task lifecycle management. The system utilizes a service-oriented architecture to decouple the user interface (CLI dispatch), business logic (Service layer), and data persistence (JSON storage). This separation ensures that the application remains maintainable and that the storage mechanism can be evolved independently of the business rules.

1.2 Maturity Assessment

The project is **READY** for execution. While a medium-severity gap exists regarding formal acceptance criteria for individual tasks, the inclusion of "Independent Tests" for each User Story provides sufficient validation gates to ensure functional correctness and successful delivery.

1.3 Technical Stack

- **Language**: Python
- **Configuration**: pyproject.toml
- **Storage**: JSON (Local File System)
- **Testing**: Unit and Integration test suites

1.4 Architectural Constraints

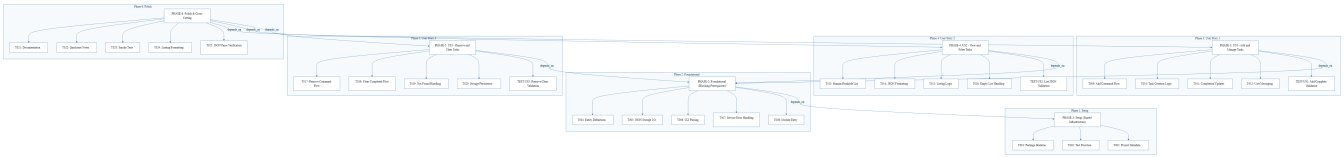
- **Persistence**: All data must be persisted in `~/.todos.json`.
- **Identity**: New tasks must follow a strict sequential ID assignment.
- **Audit**: Mandatory population of the `created_at` timestamp for every entry.
- **Output**: Support for dual output modes: human-readable and parseable JSON.
- **Execution Flow**: Strict linear dependency: Setup $\rightarrow$ Foundational $\rightarrow$ User Stories $\rightarrow$ Polish.
- **Blocking Gate**: The Foundational phase (PHASE-2) is a mandatory blocking prerequisite for all feature implementation.

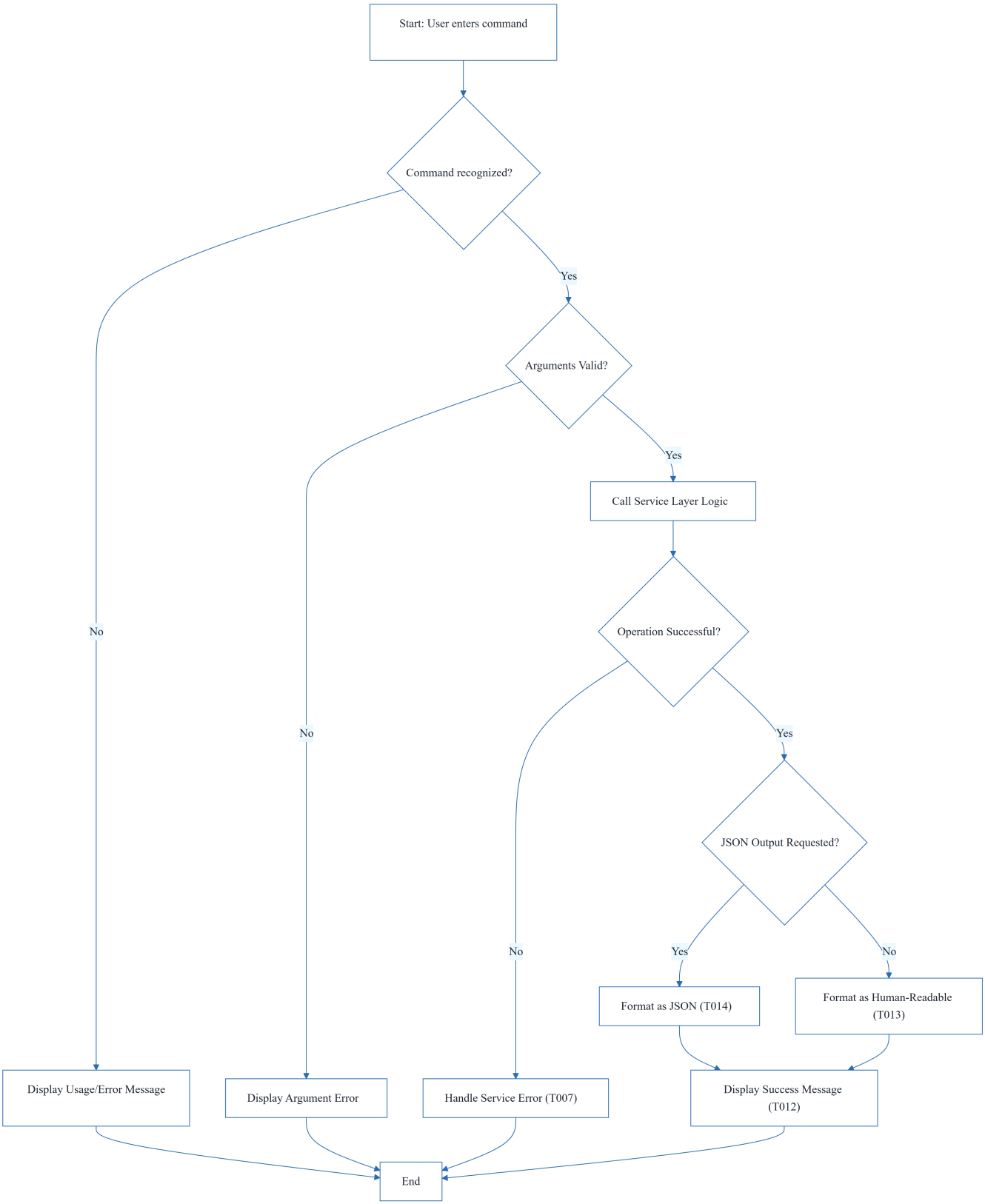
1.5 Critical Dependencies

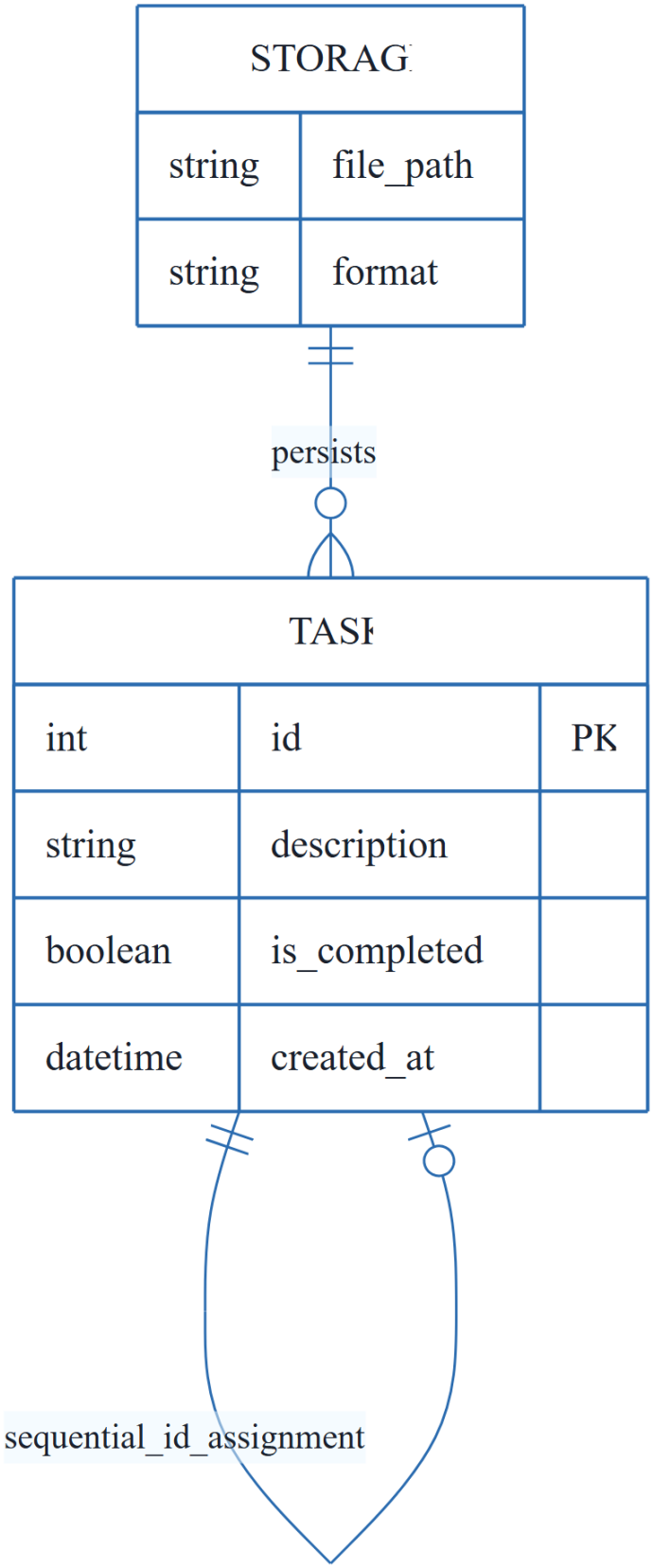
- **File System I/O**: Reliable read/write access to `~/.todos.json`.
- **ID Integrity**: Maintenance of sequential ID integrity for accurate task referencing.

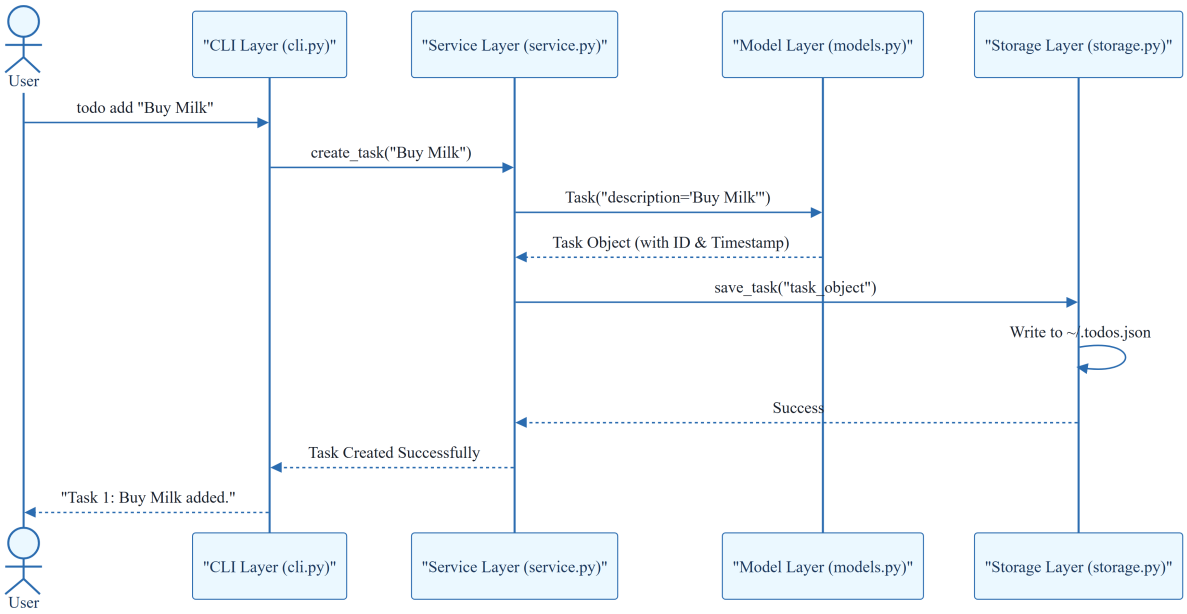
- **Phase Sequence**: PHASE-2 must be complete before US1, US2, or US3.
- **Feature Completion**: All User Stories (US1, US2, US3) must be complete before Phase 6 (Polish).
- **Internal Mapping**: Task serialization helpers in `models.py` are critical for both storage and JSON output.

2. Architecture Workflows



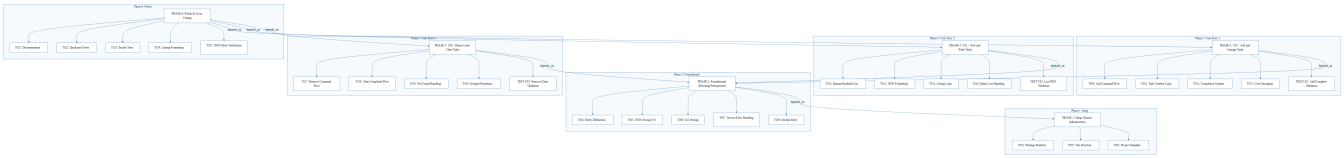

Start: User enters command
Command recognized?
No
Yes
Arguments Valid?
No
Yes
Call Service Layer Logic
Operation Successful?
No
Yes
JSON Output Requested?
Yes
No
Format as JSON (T014)
Format as Human-Readable (T013)
Display Usage/Error Message
Display Argument Error
Handle Service Error (T007)
Display Success Message (T012)
End


STORAG
string
file_path
string
format
persists
TASI
int
id
PK
string
description
boolean
is_completed
datetime
created_at
sequential_id_assignment

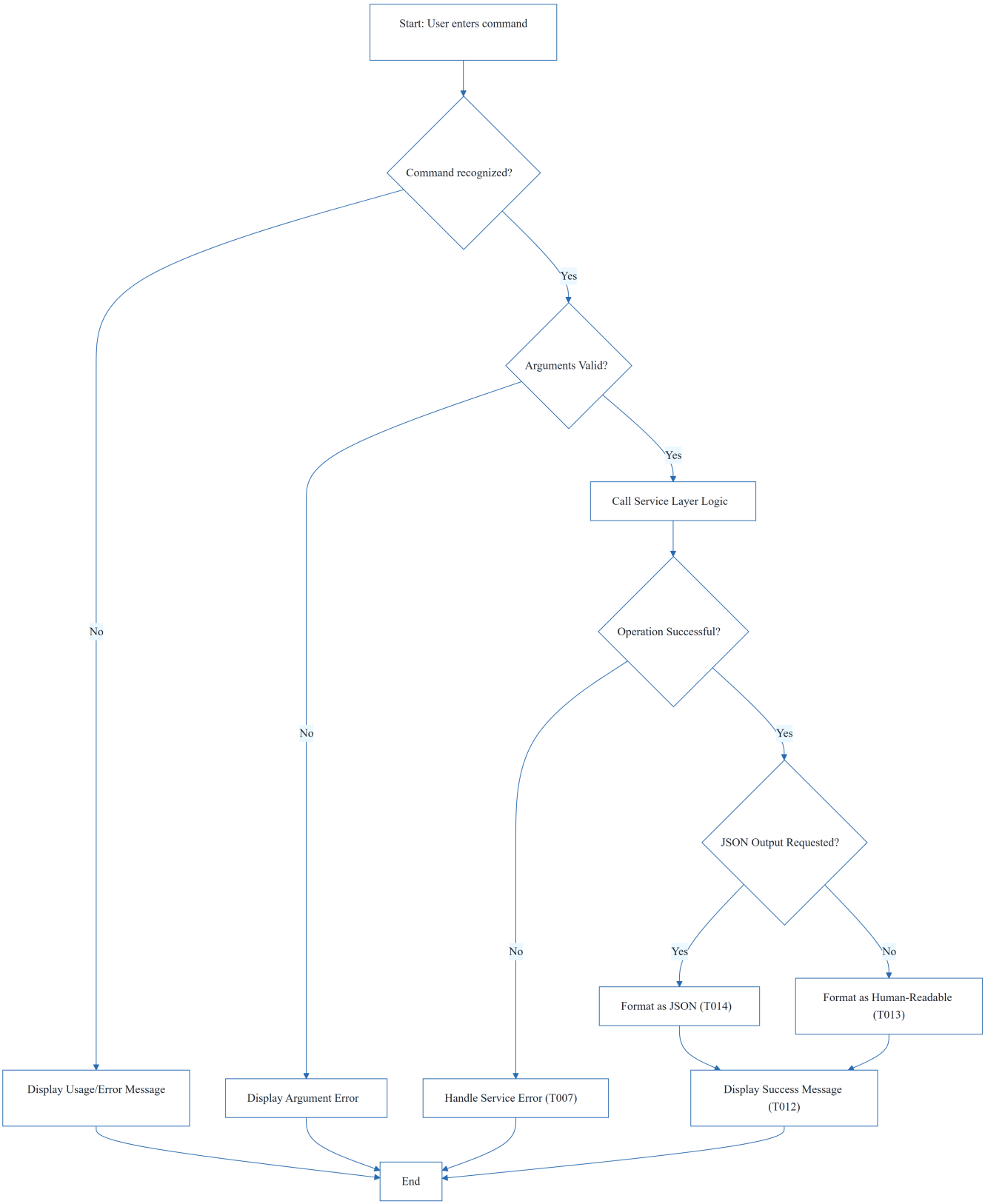


& Visual Diagrams

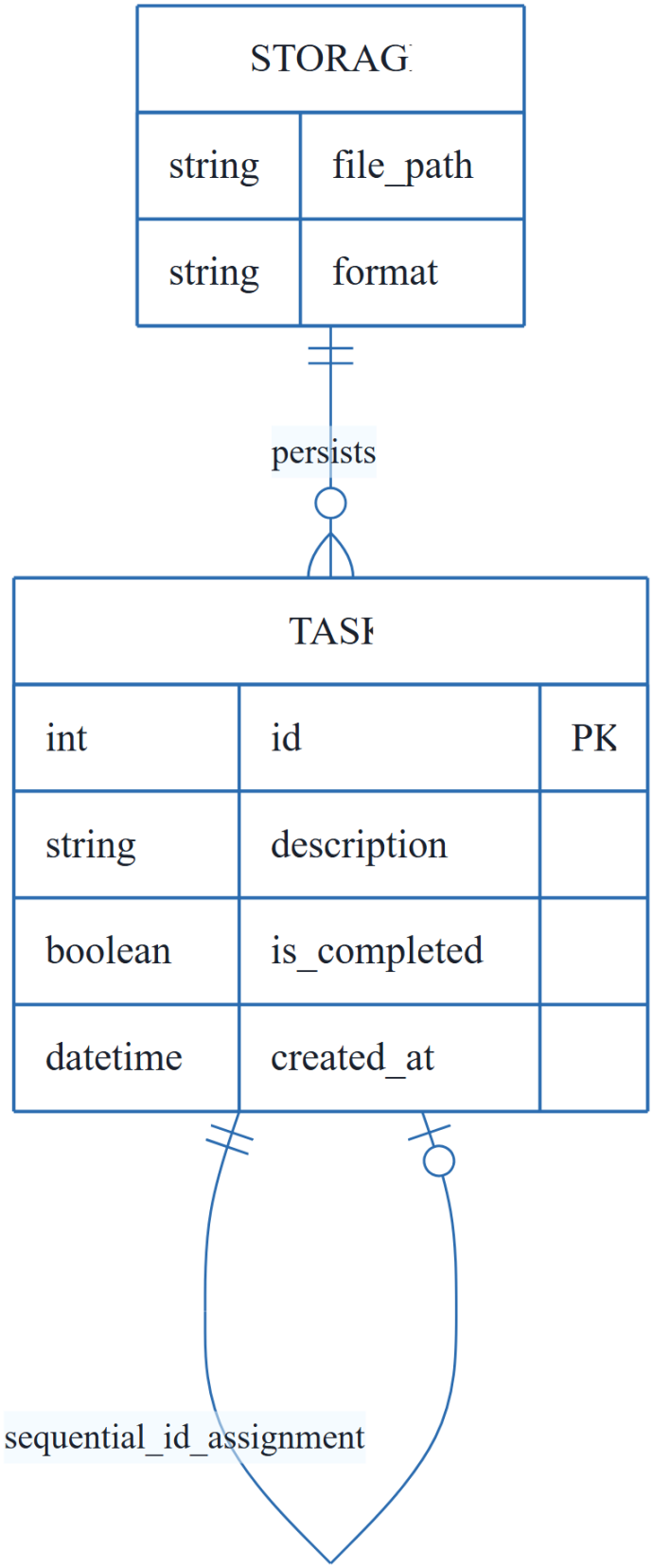
2.1 Project Implementation Traceability Map



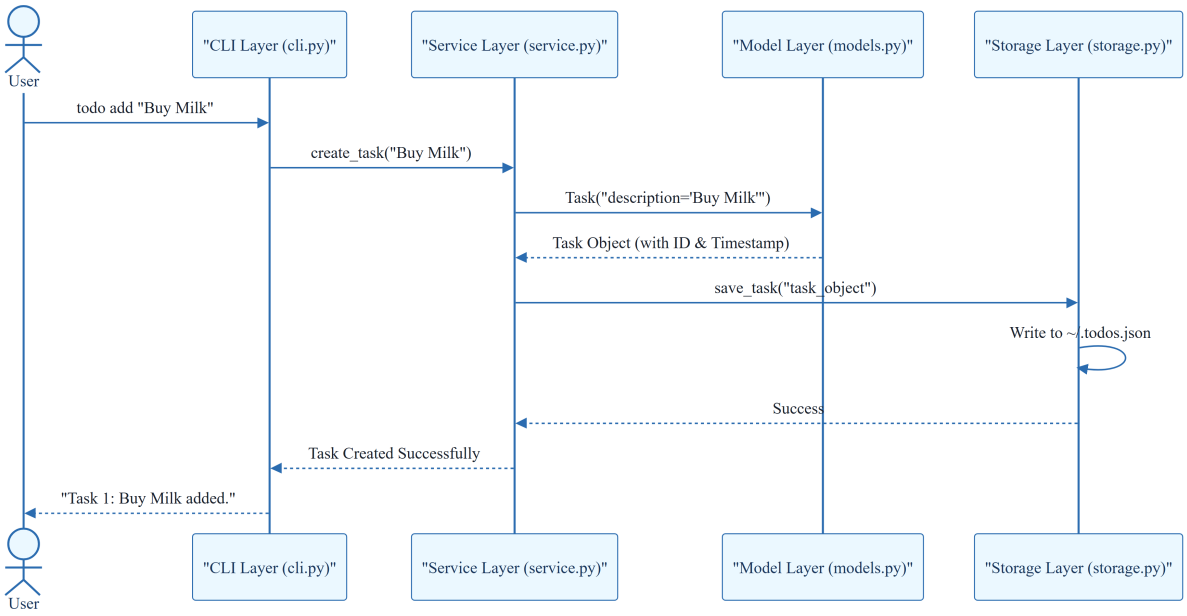
2.2 CLI Command Execution Workflow



2.3 CLI To-Do Manager Data Model



2.4 User Story Interaction Sequence



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Requirement / Task Description	Source Phase	Priority / Story
T001	Create Python package skeleton (`__init__.py`, `__main__.py`, `cli.py`, `models.py`, `storage.py`, `service.py`)	PHASE-1	Setup
T002	Create test directory structure (`tests/unit/`, `tests/integration/`)	PHASE-1	Setup
T003	Add project metadata and tooling entry points in `pyproject.toml`	PHASE-1	Setup
T004	Implement task entity definitions and serialization helpers in `models.py`	PHASE-2	Foundational
T005	Implement JSON storage path resolution and file I/O helpers in `storage.py`	PHASE-2	Foundational
T006	Implement shared command-line parsing and top-level dispatch in `cli.py`	PHASE-2	Foundational

ID	Requirement / Task Description	Source Phase	Priority / Story
T007	Implement shared service-layer error handling and task collection utilities in `service.py`	PHASE-2	Foundational
T008	Define module entry behavior for `python -m todo_manager` in `__main__.py`	PHASE-2	Foundational
T009	Implement the `add` command flow in `cli.py` and `service.py`	PHASE-3	US1
T010	Implement task creation, sequential ID assignment, and `created_at` population in `models.py`	PHASE-3	US1
T011	Implement completion updates for existing tasks in `service.py`	PHASE-3	US1
T012	Add user-facing success and error messages for add/complete in `cli.py`	PHASE-3	US1
T013	Implement human-readable task listing output in `cli.py`	PHASE-4	US2
T014	Implement `--json` output formatting in `cli.py` using `models.py` helpers	PHASE-4	US2
T015	Add listing logic that preserves task order and status fields in `service.py`	PHASE-4	US2
T016	Handle the empty-list case with a clear message in `cli.py`	PHASE-4	US2
T017	Implement the `remove` command flow in `cli.py` and `service.py`	PHASE-5	US3
T018	Implement the `clear` command flow for removing completed tasks in `service.py`	PHASE-5	US3
T019	Add not-found handling for invalid task IDs in `cli.py`	PHASE-5	US3

ID	Requirement / Task Description	Source Phase	Priority / Story
T020	Ensure storage writes persist removals and clears safely in `storage.py`	PHASE-5	US3
T021	Document CLI usage and storage behavior in `README.md`	PHASE-6	Polish
T022	Add quickstart verification notes and examples in `quickstart.md`	PHASE-6	Polish
T023	Run manual smoke test of all commands against `~/.todos.json`	PHASE-6	Polish
T024	Verify linting and formatting expectations for `src/todo_manager/`	PHASE-6	Polish
T025	Confirm generated JSON output remains parseable for `todo list --json`	PHASE-6	Polish
TEST-US1	Validation: `todo add` and `todo complete` reflect in stored list	PHASE-3	US1
TEST-US2	Validation: `todo list` and `todo list --json` output correctness	PHASE-4	US2
TEST-US3	Validation: `todo remove` and `todo clear` target correct entries	PHASE-5	US3

3.2 Security Rules

- **Data Integrity**: The system must ensure that sequential IDs are not duplicated during concurrent-like operations (though the tool is single-user CLI).
- **Input Validation**: All CLI arguments must be validated before being passed to the service layer to prevent crashes or corrupted JSON writes.
- **Error Handling**: Service-layer errors (T007) must be caught and translated into user-friendly messages in the CLI layer without leaking internal stack traces.

3.3 Data Models

- **Task Entity**:
- `id` (Integer): Primary Key, sequential.
- `description` (String): The task text.
- `is_completed` (Boolean): Completion status.

- `created_at` (DateTime): ISO 8601 timestamp.
- **Storage Entity**:
- `file_path` (String): Path to `~/.todos.json`.
- `format` (String): JSON.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	While 'Independent Tests' are provided for User Stories, formal acceptance criteria for each individual task (T001-T025) are missing.
Security & Performance Constraints	LOW	No specific security or performance constraints were mentioned; however, for a CLI tool, this is generally acceptable.
Open Questions & Uncertainties	LOW	The document appears complete with no open questions listed.

4.2 Remediation & Workflow

1. **Immediate Action**: Use the "Independent Tests" (TEST-US1, TEST-US2, TEST-US3) as the primary acceptance criteria for the associated task groups.
2. **Validation**: Perform the smoke tests (T023) as the final quality gate before project closure.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-2	A synchronization gate ensuring all blocking infrastructure is verified before parallel feature development commences.
Foundational	TECHNICAL_STACK	PHASE-2	The shared infrastructure layer containing core entity definitions, storage logic, and command dispatching that blocks all subsequent feature work.

Term	Category	Context Anchor	Project Definition
Goal	BUSINESS_DOMAIN	PHASE-3	The primary functional objective a user must achieve within a specific feature set.
ID	TECHNICAL_STACK	T010	A sequential alphanumeric token assigned to each entry to ensure unique reference and retrieval.
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage in ~/.todos.json and for machine-readable output.
MVP	BUSINESS_DOMAIN	PHASE-3	The minimum viable product consisting of the ability to create and mark entries as finished.
Organization	BUSINESS_DOMAIN	Tasks: CLI To-Do List Manager	The structural grouping of implementation units by user story to ensure independent testability.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of design documents including plan, spec, research, and data-model required before implementation.
README	TECHNICAL_STACK	T021	The primary documentation file detailing usage and storage behavior.
Setup	TECHNICAL_STACK	PHASE-1	The initial phase involving package skeleton creation and tooling configuration in pyproject.toml.
T016	TECHNICAL_STACK	T016	The specific implementation unit responsible for handling empty collection states with a user-facing message.
Tests	TECHNICAL_STACK	T002	The verification suite divided into unit and integration directories.
population in	TECHNICAL_STACK	T010	The process of assigning a timestamp to the created_at field during entity instantiation.

Term	Category	Context Anchor	Project Definition
todo clear	BUSINESS_DOMAIN	T018	The operation that removes all entries marked as finished from the persistent store.
todo list	BUSINESS_DOMAIN	T013	The operation that retrieves and displays all entries in either human-readable or machine-parseable format.
■■ CRITICAL	TECHNICAL_STACK	PHASE-2	A high-priority constraint indicating that no subsequent work can begin until the current phase is fully validated.